

CineScore+ Project Spec

A movie review and recommendation app with a leaderboard using the TMDB API.

Project Description

CineScore+ aims to be a sleek and modern version of a movie review application, with a “gamified” element in the form of a leaderboard with top reviewed movies. This leaderboard is fed data through the TMDB API, as well as users on the app through reviews. The review process is also streamlined to be a simple 3 step process to submit a review under a 5-star system. The app creates a profile for each user, which can recommend them films based on what they positively reviewed.

The app uses movie metadata (title, dates, descriptions, posters) from TMDB. Users can search for a specific film, select it from a result list, and assign it a 1-5 star rating. User ratings are stored locally and then used to create the leaderboard. Rankings are determined by the number of stars, but also the amount of reviews for each movie. Users are able to see what the community values by viewing the leaderboard.

The second major feature is the ability to recommend different films based on previous reviews. The correlation between positive ratings and genre is how the system knows how to recommend specific movies based on the user’s taste. Recommendations are created using a combination of TMDB and user review data.

For version 1, the app will run within a command line interface to ensure the basic structure is sufficient before interface design. Users are able to type commands like search, review, Movieboard (leaderboard) and recommendation. These commands will help lay out each of our major user flows and tasks before creating screens to accompany each task. Version 2 will implement a graphical user interface using Streamlit as the hosting platform.

User Flow and Vignettes

Task 1: Searching for and Submitting a Review

The user opens the CLI and sees a simple menu. They select the “search” option and type in a movie title “Idiocracy”. The system sends a request to the TMDB API and returns a list of matching movies to the user search term. The user selects a movie to review by entering its ID number displayed next to the search result. The system prompts the user to select a

rating between 1 and 5 representing the 5-star system. Once submitted, the rating is stored locally, and the movie's overall rating is updated on the leaderboard.

Technical details:

- TMDB endpoints: /search/movie, /movie/{id}
- Data stored JSON: {movie_id, rating}
- Validate rating output (1-5 stars only)
- Cache movies details to reduce API calls
- CLI flow: search → select → rating

Task 2: Viewing Movieboard

The user selects “View Movieboard” from the first menu on the CLI. The system loads all the stored reviews, groups them by the movie ID, and shows the average rating of each movie, and the number of ratings. The system then sorts movies by average rating (descending) and displays the top 10. Each entry on the board includes the title, average rating, and the total reviews. The user can choose to filter the leaderboard by genre if they desire (e. i., “Action”, “Horror”).

Technical details:

- Average rating formula = $\text{sum}(\text{ratings}) / \text{count}(\text{ratings})$
- Sorting that needs to be performed in Python
- Genre filtering that uses TMDB genre IDs
- Data structure: a dictionary keyed by movie_id
- CLI output: numbered list of top-rated movies

Task 3: Getting a Recommendation

The user selects “What should I watch?”. The system reviews their rating history to determine which genres they rate the highest. It then gets trending or popular movies from TMDB and filters them by the user's highest rated genre. The user receives a selection from the top ten filtered results of this genre. The title, year and description is shown in CLI, with the poster being a graphical addition that can be made in Version 2.

Technical details:

- Identify top genre from user's past ratings
- TMDB endpoints: /trending/movie/week, /genre/movie/list
- Genre based filtering for recommendation algorithm
- Possible future addition: similarity scoring

- CLI flow: compute → fetch → filter → display

Back End Flow

Data

- reviews.json
 - Stores all user ratings and movie IDs
- movie_cache.json
 - Stores TMDB metadata locally to not repeat API calls
- Leaderboard structure

```
{  
    movie_id: {  
        "title": "...",  
        "ratings": [4, 5, 3],  
        "avg": 4.0,  
        "review_count": 3  
    }  
}
```

- user_profile
 - Stores user's genre preferences

Core Functions

def search_movie(search_term):

Calls TMDB search endpoint

Returns list of movie dictionaries

def get_movie_details(movie_id):

Gets metadata from TMDB API or local cache

def submit_rating(movie_id):

Validates user rating and stores it for later use

Updates leaderboard

```
def generate_movieboard(entries=20, genre_filter=None):
```

```
    # Finds average reviews and sorts movies
```

```
def recommend_movies(user_history):
```

```
    # Computes genre preferences
```

```
    # Gets trending movies
```

```
    # Filters by preferred genres
```

Program Flow

1. User Input (CLI)

- a. Commands: search, review, movieboard, recommendation
- b. Input: movie title, review, genre filter

2. TMDB API Layer

- a. Sends requests
- b. Receives JSON responses
- c. Stores results in local cache

3. Local Storage Layer

- a. Reads and writes reviews.json
- b. Updates averages

4. Processing Layer

- a. Creates Movieboard
- b. Gives recommendations
- c. Formats output

5. Output Layer (CLI)

- a. Displays lists, movie details and prompts

Data types involved:

- Lists (search results, reviews)
- Dictionaries (metadata, movieboard entries)
- JSON files (data storage)
- Strings (titles, descriptions)
- Integers (reviews, IDs)

Final Self-Assessment

The biggest change I had to make when putting this spec together was the affection rating. I think keeping the system simple is a good rule of thumb, at least for version 1. Using the simple 5-star rating across the board creates a simpler flow for users and for when I must translate that flow into code, especially for the recommendation system. I feel confident with this spec now that everything is laid out. The TMDB API seems very detailed and much more than what I need for the purposes of this project but hopefully leaves some room for future improvements. The biggest potential issue I have currently is ensuring that the API works correctly while being within my application, which I am sure I will pick up when implementing version 1, but from tests I have done I have confidence that it will, despite possible errors that could break user flow if the API has issues pulling data. I am least familiar with utilizing an API, so if there is any aspect of the project I may need the most help in it is there. The demo of TMDB was very useful. Overall, I am ready to see how this project fleshes out.